NAAN MUDHALVAN PROJECT

INTRODUCTION:

PROJECT TITLE: SPENDWISE

TEAM LEADER NAME:p.vasundhra

EMAIL ID: .vasundrap363@gm

TEAM ID: SWTID-2026-6042

TEAM MEMBERS:

NAME:	EMAIL ID :
A.mithra	mithraa251@gmail.com
V.sathya	sathyasathyavenkatesh@gmail.com
P.gopika	gopikasumanth@gmail.com
M.pavithra	pavithrapavi0763@gmail.com

Project Overview

SpendWiseisabackend application designed to help beginner developers learn how to build secure RESTAPIsusing Node.js,Express, MongoDB, and JWT authentication. The project focuses onimplementing user authentication and basic CRUD operations in a clean and simple waysostudents can understand how backend systems work in real-world scenarios.

Key Objectives:

- Provide secure user authentication using JWT
- Enable basic CRUD operations on user data (transactions/records)
- Demonstrate MongoDB integration using Mongoose
- Show how protected routes work using middleware
- Maintain a simple and beginner-friendly backend architecture

Target Users:

- Beginner backend developers
- Students learning Node.js and Express
- Freshers preparing for technical interviews
- Anyone who wants a simple project to understand JWT and MongoDB
- Learners who want hands-on practice with REST APIs

Project Instruction

SpendWiseisabackend-onlyproject built using Node.js and Express.

Quick Start:

```
npm init

npm install

npm run dev
```

Steps to Create the Project:

- Choose or create a directory for the backend project.
- Open terminal and navigate to that directory.
- Initialize Node.js project using npm init.
- Install required packages: express, mongoose, jsonwebtoken, bcrypt, cors, dotenv.
- Create folders: models, routes, controllers, middleware, config.
- Start the server using npm run dev.
- Test APIs using Postman or Thunder Client at http://localhost:5000.

Pre-requisites:

Software Requirements

Software	Purpose
Node.js	JavaScript runtime for backend
MongoDB	NoSQL database for data storage
npm	Package manager
Git	Version control system
Code Editor	VS Code or similar

Knowledge Requirements

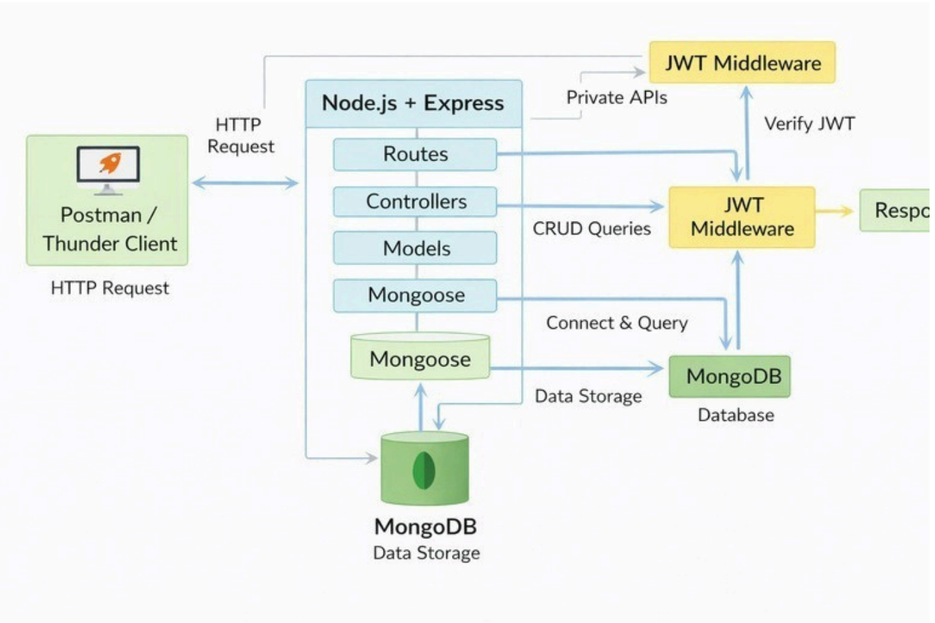
- JavaScript ES6+
- Node.js and Express.js
- MongoDB and Mongoose
- JWT authentication
- Basic REST API concepts
- Git basics

Thumbnail for Project

- A simple backend project thumbnail representing:

Node.js + Express + MongoDB + JWT + CRUD APIs

Project Architecture Image



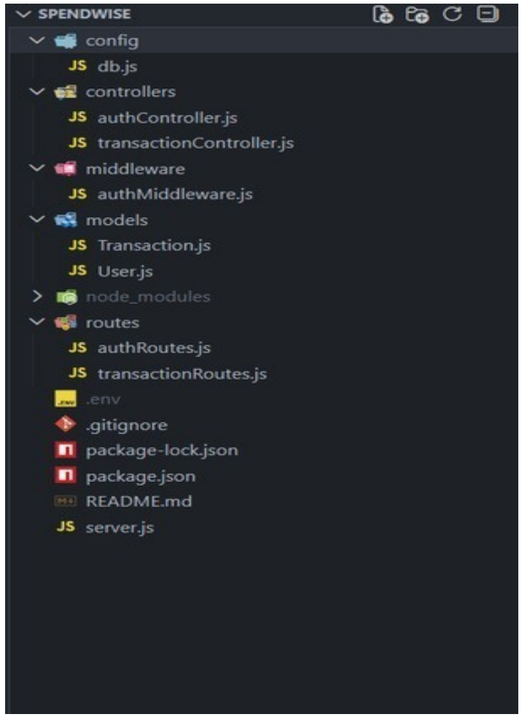
Technical Architecture

BackendArchitecture(Node.js+ Express)

- **Controllers:** Handle business logic for authentication and CRUD
- **Models:** Mongoose schemas for User and Transaction
- **Routes:** APIendpoints for auth and CRUD operations
- **Middleware:** JWT verification and request protection
- **Configuration:** Database connection and environment variables

Database Architecture (MongoDB)

- Users Collection: `_id,name,email,password(hashed), createdAt`
- Transactions Collection: `_id,userId(FK),title/description, amount/value, createdAt`
- Relationship: Many Transactions belong to One User



Project Setup

1. Install required tools:

- Node.js
- MongoDB

2. Create project folders:

- server folders (models, routes, controllers, middleware, config)

3. Install Backend Packages:

- Express
- Mongoose
- Cors
- Jsonwebtoken
- Bcrypt
- Dotenv

BACKEND DEVELOPMENT

- Setup Expressserver
- Create server.js file
- Create .env file and define PORT and DB URL

```env
PORT=5000
MONGO_URI=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
JWT_SECRET=your_secret_key
```

- Configure middleware (cors, express.json)
- User Authentication:
  - Create Register and Login routes
  - Generate JWT token
  - Protect routes using middleware

- Define API Routes:
  - Auth routes

```jsx
const router = express.Router();

router.post("/register", registerUser);
router.post("/login", loginUser);

module.exports = router;
```

  - Transaction CRUD routes

- Implement Data Models:

```jsx
const transactionSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    amount: {
      type: Number,
      required: true,
    },
    type: {
      type: String,
      enum: ["income", "expense"],
      required: true,
    },
    category: {
      type: String,
      default: "General",
    },
    note: {
      type: String,
    },
    date: {
      type: Date,
      default: Date.now,
    },
  },
  { timestamps: true }
);
```

- Implement CRUD operations
- Add basic error handling

DATABASE DEVELOPMENT

Configure MongoDB

- Use MongoDB Atlas or local MongoDB
- Store connection URL in .env file
- Connect using Mongoose

Create Database Connection

- Create a config file for DB connection

```javascript
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

config > JS db.js > ...

```javascript
const mongoose = require("mongoose");

const connectDB = async () => {
    try {
        const conn = await mongoose.connect(process.env.MONGO_URI);
        console.log(`MongoDB Connected: ${conn.connection.host}`);
    } catch (error) {
        console.error("MongoDB connection error:", error.message);
        process.exit(1);
    }
};

module.exports = connectDB;
```

- Use `mongoose.connect()`
- Handle success and error cases

Create Schema and Models

User Model:

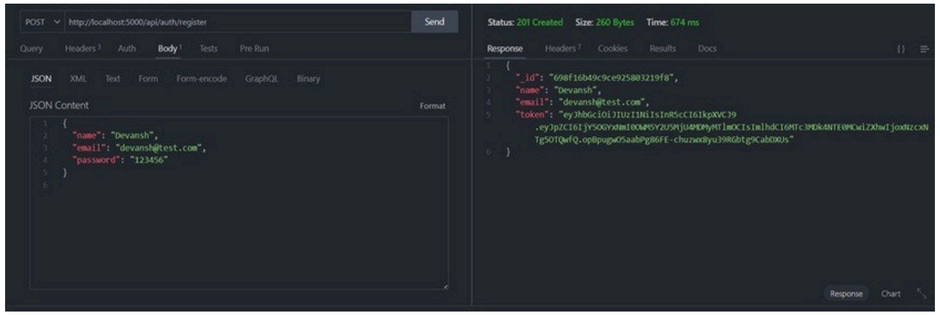
- Name
- email
- password

Data / Transaction Model:

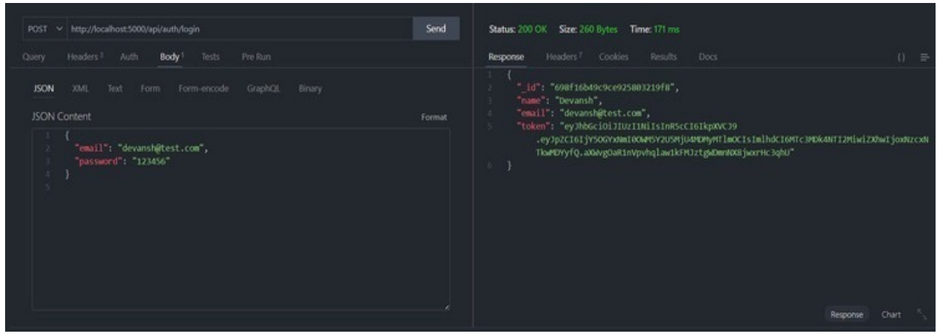
- userId
- title / description
- amount / value
- timestamps

PROJECT EXECUTION

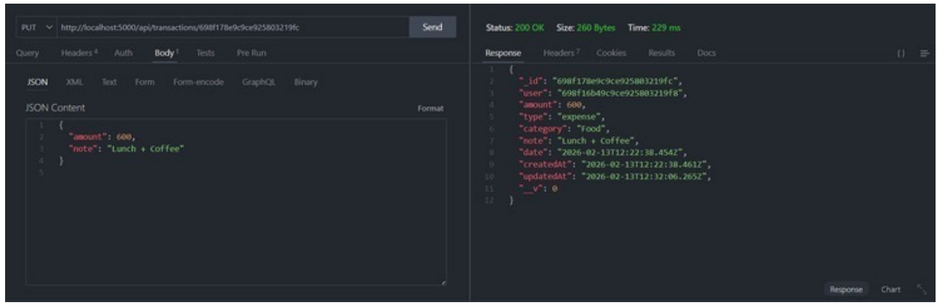
Project Execution Steps:



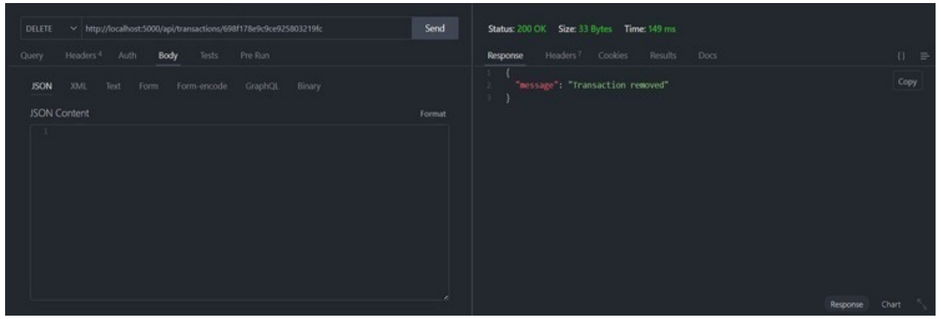
1. Login API



1. Create Transaction



1. Delete API



Project Demo Video

Drive Link : SpendWise